

- Description

This module verifies whether a given withdrawal amount is valid based on two conditions:

1. The withdrawal amount **must be greater than zero**.
2. The withdrawal amount **must not exceed the account balance**.

If both conditions are met, the withdrawal is considered valid; otherwise, it is rejected.

- Specification

module ATMValidation

var amount, accountBalance: real

process isValidWithdrawal(amount: real, accountBalance: real) flag: boolean

functional scenario form:

amount <= accountBalance && flag =true, Valid withdrawal, amount meets all conditions

||

amount > accountBalance && flag = false, Invalid withdrawal, insufficient account balance

||

amount <= 0 && flag = false, Invalid withdrawal, amount must be greater than zero

||

accountBalance<= 0 && flag = false, Invalid withdrawal, balance must be greater than zero

end process

end module

- Java Code

```
package code;
public class c02_ATMValidation {

    public static boolean isValidWithdrawal(double amount, double accountBalance) {
        boolean flag;
        if(amount > 0 && accountBalance > 0 && amount <= accountBalance) {
            flag = true;
        }
        else flag = false;
        return flag;
    }

    public static void main(String[] args) {
        // testcase-VT:
        double[][] testCases = {
            {2598.16, 1784.78}, //amount > 0 && accountBalance > 0 && amount <= accountBalance
        }
    }
}
```

```

        {197.64, 102.47}, //amount > 0 && accountBalance > 0 && amount > accountBalance
        {1015.07, 4908.75}, //amount > 0 && accountBalance > 0 && amount <= accountBalance
        {-
1939.66, 2509.62}, //amount <= 0 && accountBalance > 0 && amount <= accountBalance
        {1707.62, -
753.75}, //amount > 0 && accountBalance <= 0 && amount > accountBalance
    };
    // testcase-FT:
    // double[][] testCases = {
    //     {-1.7976931348623157E308, 0},
    //     {4.565679614340899E293, 0},
    //     {0, 0},
    //     {-6.4290438513570355E301, -1.7976855920277024E308},
    //     {7.714599345286304E297, 3.313395189713529E307},
    // };

    // testcase-Z3:
    // double[][] testCases = {
    //     {200, 500},
    //     {600, 400},
    //     {0, 300},
    //     {100, 0},
    //     {300, 300},
    // };

    // UVT
    // double[][] testCases = {
    //     {2598.16, 1784.78},
    //     {197.64, 102.47},
    //     {1015.07, 4908.75},
    //     {1939.66, 2509.62},
    //     {1707.62, 753.75},
    // };

    for (double[] testCase : testCases) {
        boolean result = isValidWithdrawal(testCase[0], testCase[1]);
        System.out.printf("isValidWithdrawal(%.1f, %.1f) = %b\n",
            testCase[0], testCase[1], result);
    }
}

```

```
}
```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	808	816
▼ src	1.0 %	8	808	816
▼ code	1.0 %	8	808	816
▼ c02_ATMValidation.java	100.0 %	8	0	8
▼ c02_ATMValidation	100.0 %	8	0	8
\$ isValidWithdrawal(double, double)	100.0 %	6	0	6
\$ main(String[])	100.0 %	2	0	2

3. Vibration testing instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	137	15,373	15,510
▼ src	0.9 %	137	15,373	15,510
▼ code	0.9 %	137	15,373	15,510
▼ c02_ATMValidation.java	97.9 %	137	3	140
▼ c02_ATMValidation	97.9 %	137	3	140
\$ isValidWithdrawal(double, double)	100.0 %	19	0	19
\$ main(String[])	100.0 %	118	0	118

4. Fuzz testing branch coverage: 87.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.9 %	7	809	816
▼ src	0.9 %	7	809	816
▼ code	0.9 %	7	809	816
▼ c02_ATMValidation.java	87.5 %	7	1	8
▼ c02_ATMValidation	87.5 %	7	1	8
\$ isValidWithdrawal(double, double)	83.3 %	5	1	6
\$ main(String[])	100.0 %	2	0	2

5. Fuzz testing instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	137	15,373	15,510
▼ src	0.9 %	137	15,373	15,510
▼ code	0.9 %	137	15,373	15,510
▼ c02_ATMValidation.java	97.9 %	137	3	140
▼ c02_ATMValidation	97.9 %	137	3	140
\$ isValidWithdrawal(double, double)	100.0 %	19	0	19
\$ main(String[])	100.0 %	118	0	118

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	832	840
▼ src	1.0 %	8	832	840
▼ code	1.0 %	8	832	840
▼ c02_ATMValidation.java	100.0 %	8	0	8
▼ c02_ATMValidation	100.0 %	8	0	8
\$ isValidWithdrawal(double, double)	100.0 %	6	0	6
\$ main(String[])	100.0 %	2	0	2

7. Z3 instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	137	15,373	15,510
▼ src	0.9 %	137	15,373	15,510
▼ code	0.9 %	137	15,373	15,510
▼ c02_ATMValidation.java	97.9 %	137	3	140
▼ c02_ATMValidation	97.9 %	137	3	140
\$ isValidWithdrawal(double, double)	100.0 %	19	0	19
\$ main(String[])	100.0 %	118	0	118

8. Unimproved V-method branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	834	840
▼ src	0.7 %	6	834	840
▼ code	0.7 %	6	834	840
▼ c02_ATMValidation.java	75.0 %	6	2	8
▼ c02_ATMValidation	75.0 %	6	2	8
isValidWithdrawal(double, double)	66.7 %	4	2	6
main(String[])	100.0 %	2	0	2

9. Unimproved V-method instruction coverage: 97.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	137	21,949	22,086
▼ src	0.6 %	137	21,949	22,086
▼ code	0.6 %	137	21,949	22,086
▼ c02_ATMValidation.java	97.9 %	137	3	140
▼ c02_ATMValidation	97.9 %	137	3	140
isValidWithdrawal(double, double)	100.0 %	19	0	19
main(String[])	100.0 %	118	0	118